Design and Analysis of Algorithms
Taubakabyl Nurlybek
Assignment 4 - Dynamic Programming and String Algorithms
Deadline: 08.11.2026 23:59

Submission: ZIP archive named DAA_Assignment4_name_surname_group.zip
- Upload to Moodle.
- Include a link to your GitHub repository (branch main, tag v1.0).

Scenario: City Search & Spell-Check

The city platform gets a search box. Residents type street names with typos ("Abylai Han" instead of "Abylai Khan"), and operators look for error patterns in very long log files. Two small questions come out of this: how different are two words, and where does a pattern appear in a long text?

Both questions have classic answers from Lectures 8 and 9: dynamic programming and string matching. This is the last assignment, so there is less code than before. The main work is yours: you decide how to test the algorithms, which sizes to use, what to measure and which plots tell the story.

In Assignment 4, you will:

- Edit Distance in three ways and Naive and KMP string search.
- Check your code with small tests and reference values.
- Design your own experiment: input sizes, input types, metrics and plots.
- Explain Implement in a short report what you expected, what you saw and why.

Learning Objectives

- Describe a DP solution in three lines: state, transition and base case.
- See why plain recursion repeats work and how a table (or two rows) removes it.
- Explain how the KMP prefix function (LPS) avoids re-checking characters.
- Choose inputs that show the best and the worst case of an algorithm, not only the average one.
- Choose metrics and plots that give a clear answer to a clear question.
- Write conclusions that match your data.

System Requirements

1. Edit Distance (Dynamic Programming)

Edit distance is the smallest number of insertions, deletions and replacements that turn string a into string b (Lecture 8, slide 8). Write one method int distance(String a, String b) in three versions:

- **A. Recursive:** Direct recursion on the last characters, no memory. It is slow on purpose.
- **B. Table:** Bottom-up, a (n+1) x (m+1) table, time O(n·m).

- **C. Two rows:** The same answer from two rows only, so the extra memory is O(m) instead of O(n·m).
- **DP Design in Comments:** Above each version, write the state, the transition and the base case in 2-3 lines.
- **Edge Cases:** Empty strings must work. All three versions must return the same number on every input.

2. String Search: Naive & KMP

Both methods take a text T and a pattern P and return the start indices of **all** occurrences, in increasing order, including overlapping ones. If there is no match, return an empty list.

- **Naive:** Try every start position and compare character by character (Lecture 9, slide 4).
- **KMP:** Build the LPS array first (int[] lps(String p) must be public so that tests can call it), then search without moving backwards in the text.
- **Rules:** An empty pattern throws IllegalArgumentException. A pattern longer than the text returns an empty list.

3. Counters

Wall-clock time depends on the laptop, so you also need counters. A counter is a number that your algorithm increases while it works, for example every time it does one unit of work that you consider important. Rules:

- **You decide what to count.** Pick counters that match the cost you want to explain, and say why in the report.
- **Count inside the algorithm.** A number computed from a formula afterwards is not a measurement.
- **At least one counter per algorithm family** (Edit Distance, string search), and also the time (median of 3-5 runs).

4. Your Experiment (you design it)

Here is the main part of the assignment. There is no fixed list of sizes, inputs or plots. Fill in the **Experiment Design Card** below **before** you run anything, and keep it at the start of your report. You may improve it later, but then write what you changed and why.

Card field	**What to write**
Question	One sentence per algorithm family: what do you want to find out? (For example: does the cost grow the way the theory says?)
Sizes (n)	The list of sizes you will use and why. Sizes may differ between algorithms.
Inputs	The kinds of input you will generate (random, special, best case, worst case...) and what each one is supposed to show.

Card field	What to write
Metrics	The counters and the time you will record, and the reason for each.
Prediction	What you expect to see, in words or as a rough sketch of the curve. Write it before the first run.
Plots	The plots you plan to draw and the question each plot answers.

Minimum rules (everything above these rules is your choice):

- **Sizes:** At least 4 different values of n for each algorithm. The largest should be at least 4 times the smallest. Choose sizes that your algorithm can really handle: the clock should not show 0 ms everywhere, and nothing should run for an hour.
- **Inputs:** At least 2 input types for Edit Distance and 2 for string search, and at least one of them should be chosen to make the algorithm work as hard as possible.
- **Reproducible:** Write your own generator with a fixed seed. The same command must give the same numbers.
- **Plots:** At least 3 plots in total. Each plot has a title, labeled axes with units and one sentence under it that says what the reader should notice. At least one plot must use a log axis or a ratio.
- **Results file:** results.csv with one row per run and your own columns. At least algorithm,input,n,time_ms plus your counters.

5. Tests (JUnit 5)

- **Edit Distance:** The reference values from section 7, plus empty strings. On 100 random short strings the three versions give the same number.
- **LPS:** The reference arrays from section 7.
- **Search:** The reference matches from section 7, including overlapping matches. On 100 random texts Naive and KMP return the same list.
- **Edge Cases:** Empty pattern throws, pattern longer than the text returns an empty list.

6. Report (REPORT.md or PDF, 2-4 pages)

- **Experiment Design Card** (section 4), at the start.
- **Plots and Results:** Your plots, each with its one-sentence takeaway.
- **Prediction vs Result:** For each family, what you expected, what you saw and, if they differ, why.
- **Short Answers (3-5 sentences each):**
  - Which input made your string search slowest, and why does it do that?
  - How large an n could the recursive Edit Distance handle, and what stopped it?
  - What do you gain and what do you lose when you use two rows instead of the full table?

    - If you had to choose one string search for your city platform, which one and why?
- **One Paragraph on Time:** What your measured times looked like and why they are not your only evidence.

7. Reference Values (check yourself)

Your code must give exactly these results. Positions start from 0.

Input	Expected result
distance("kitten", "sitting")	3
distance("flaw", "lawn")	2
distance("intention", "execution")	5
distance("", "abc") and distance("abc", "abc")	3 and 0
lps("AAACAAAA")	[0, 1, 2, 0, 1, 2, 3, 3]
lps("ABABCABAB")	[0, 0, 1, 2, 0, 1, 2, 3, 4]
search("ABABDABACDABABCABAB", "ABABCABAB")	[10]
search("AAAAA", "AA")	[0, 1, 2, 3]
search("ABABABA", "ABA")	[0, 2, 4]

8. Git

- **Branches:** Work on main. Tag the final version v1.0.
- **Commits:** Small steps with clear messages, e.g. feat(edit): add two-row version, test(kmp): add overlap case, docs(report): add plots.
- **Layout:** src/main/java, src/test/java, results/results.csv, results/plots/, README.md, REPORT.md.

Design Quality Standards

Best-Case Design (Target)

- **One Interface Per Family:** The three Edit Distance versions share one method signature, so the experiment is one loop.
- **Input With a Purpose:** Every input type in your experiment has a sentence that says what it is supposed to show.

- **Prediction First:** The prediction is written before the first run, so the plots can confirm it or surprise you.
- **Readable Plots:** A stranger can read each plot without asking you what the lines mean.

Worst-Case Design (Avoid)

- **Only Random Input:** It can hide the worst case completely, and you will draw the wrong conclusion.
- **All Points Near Zero:** Sizes so small that every time is 0 ms and every plot is a flat line.
- **Waiting for Hours:** Running the recursive version on a size it cannot survive.
- **Conclusions From One Run of the Clock:** "Version B is faster" measured once, with no counter behind it.
- **Plots Without Axes or Units:** If the reader has to guess, the plot does not count.

Deliverables

1. **Source Code:** Maven or Gradle project with the Edit Distance versions, Naive, KMP, your counters, your generator and JUnit 5 tests.
2. **results.csv:** All runs of your experiment, with your own columns.
3. **Plots:** At least three PNG charts in results/plots/.
4. **Report:** REPORT.md or PDF with the Experiment Design Card, the plots, prediction vs result and the short answers.
5. **README:** How to build the project, run the tests and run the experiment, plus your AI-use acknowledgement.

Grading Rubric

Criterion	Weight	What is evaluated	High-Performance Indicators
Edit Distance	25%	Three versions of the DP	All three agree on every input; base cases and empty strings; state, transition and base case written in comments.
String Search	25%	Naive and KMP	All matches found, including overlapping ones; correct LPS; no backward moves in the text for KMP.
Experiment Design	20%	The Experiment Design Card	Card filled in before the runs; sizes, inputs and metrics are justified; at least one worst-case input; reproducible generator.

Criterion	Weight	What is evaluated	High-Performance Indicators
Plots & Conclusions	20%	Plots, prediction vs result, short answers	Readable plots with takeaways; conclusions match the data; honest explanation of any surprise.
Tests & Repo	10%	JUnit 5 tests, README, Git history	Reference values pass; Naive and KMP cross-checked; builds with one command; tag v1.0.